

Game Quality Hub Demo — 需求与迭代说明

我怎么理解这个需求

拿到这几道题时，我的第一反应不是把它们看成几个彼此独立的小工具。

玩家反馈、Bug 分诊、测试执行、版本提测、测试资源，本身其实就在 QA 日常工作的同一条链路里。

QA 的上游可能是玩家、客户或者内部团队反馈的问题。收到这些信息以后，需要先判断哪些是真正值得处理的问题，再把它们分发给对应的人。研发解决以后，QA 还要重新验证，确认问题是不是真的被解决。到了版本发布之前，又需要通过测试和检查来判断这个版本的质量。设备和账号这些资源，本质上则是完成这些测试工作时需要的物料。

所以我没有把目标定义成“做五个工具”，而是想先把这条质量管理的主线串起来。

其中我最先深入拆的是玩家反馈这一段。

所有被系统收进来的原始内容，我把它定义成 Case。Case 首先解决的是“准确记录事实”：用户说了什么、来自哪里、什么时候发生。

但并不是每一条 Case 都值得进入后续处理流程。真正有价值、需要团队讨论或者处理的问题，才会进一步形成 Issue。Issue 再根据实际情况被判断成 Bug、Feature 或者其他类型。

这个区分对我来说很重要。

Case 是原始信息和证据，Issue 才是团队真正需要处理的议题。很多玩家可能都在反馈同一个问题，所以多条 Case 最后完全可能只对应一个 Issue。这样系统关注的就不只是“收到了多少条反馈”，而是“现在到底有哪些问题正在发生”。

我在设计过程中一直用一个比较简单的判断方法决定做什么、不做什么：

先定义这一阶段到底要解决什么问题，然后只做让最小业务闭环成立所必须的东西。

如果缺少某个能力，这条业务链就走不下去，那它就是这一轮必须做的。如果没有它业务仍然可以完整跑通，只是体验没那么好、效率没那么高，或者智能化没那么强，那就可以暂时放到后面。

所以第一轮里，我更关心 Case 能不能被记录、判断、归类，并进入 Issue；Issue 能不能被分类、指派和继续跟进。相比之下，更复杂的自动化、真实外部系统接入或者更完整的生产能力，都不是这一阶段最重要的问题。

这次我刻意没有做什么

这次我没有接真实的 Reddit、Discord 或其他玩家反馈渠道，也没有接真正的 AI 模型。现在这些能力主要是通过本地数据和模拟逻辑来表现。

原因比较简单：这一轮我想先确认“信息进来以后应该怎么流转”，而不是先花大量时间解决“怎么把信息抓进来”或者“模型到底够不够准”。

我也没有做真正的数据库、账号权限和多人协作。对于一个正式产品，这些当然都是必要能力，但对于这一轮 Demo 来说，它们并不会改变我最想验证的业务关系，所以先用本地状态把链路跑通。

业务上我也刻意没有把所有流程一直做到最后。

现在已经可以从玩家反馈走到 Case，再形成 Issue，也可以管理版本、测试、Checklist 和资源。但真正从“测试失败”继续进入 Issue、修复后重新验证，再影响最终版本发布判断，这条链还没有完全闭合。

这个部分不是没想到，而是我认为一旦继续往下做，就会同时进入 Bug 生命周期、Build、重新测试和发布决策等更大的范围。第一轮如果全部展开，很容易为了“功能完整”而把最核心的业务关系做得反而不清楚。

如果再做一轮，我会先加什么

如果再做一轮，我不会优先去增加更多页面，也不会第一时间接一个更强的 AI。

我会先把目前这条链里最明显的断点补上：

测试失败 → 创建或关联 Issue → 问题修复 → 新 Build → 重新验证 → 版本质量判断。

现在系统已经能够记录一次测试是通过、失败、阻塞还是未执行。下一步更重要的是：当一次测试失败以后，它应该能够进入真正的问题处理流程；问题被修复以后，QA 又能够清楚地知道自己是在验证哪个修复、在哪个 Build 上验证、修复前后结果发生了什么变化。

做到这一步以后，“已修复”就不只是一个人为修改的状态，而是能够找到对应的测试证据。

再往后，我才会考虑把 Version、Checklist、Issue 和 Test Run 这些信息进一步汇总成 Release Gate，帮助版本负责人判断还有哪些风险、哪些关键问题没有关闭、核心测试是否已经完成。

我认为这个项目下一步最重要的事情，不是继续增加更多功能，而是把已经存在的这些对象真正连成一个完整的质量闭环。

第一轮我想证明的是：

这些质量信息之间应该是什么关系。

如果再做一轮，我希望证明的是：

这条关系能不能真正支撑一个问题从被发现，到被解决、被验证，再到版本最终发布。